

Computer Architectures

Exam of 03/02/2026

B2

Please read the following notes before proceeding

- 1) It is recommended to save the project on the C drive during development. Once you have completed your work, you must save the project inside the exam folder on the Z drive (the same folder where you found this document). **Projects saved elsewhere will not be collected and will be permanently lost.**
- 2) The assembly subroutine developed in the first exercise must comply with the ARM Architecture Procedure Call Standard (AAPCS) standard (in terms of parameter passing, returned value, callee-saved registers).
- 3) If you prefer, you can begin with the second exercise. Since it requires calling the assembly subroutine from the first exercise, you can create a stub in the assembly file to proceed. For example:

```
subroutineName    PROC
    ; to do: add code
    MOV r0, #10    ; example of a return value
    BX LR
    ENDP
```

- 4) To earn points on all exercises, it is recommended that you write at least some code for each.
- 5) **The final project must run on the actual board (e.g., button debouncing is required).**

Special characters with Italian keyboards

- Square brackets []
Left bracket: ATL GR + [
Right bracket: ALT GR +]
- Curly brackets { }
Left bracket: LSHIFT + ALT GR + [
Right bracket: LSHIFT + ALT GR +]
- Hash #: ALT GR + à

Bitwise operations in C

Left shift by N bits = $A \ll N$
Right shift by N bits = $A \gg N$

A XOR B = $A \wedge B$
A AND B = $A \& B$ (&& is the logical AND)
A OR B = $A | B$ (|| is the logical OR)

Computer Architectures

Exam of 03/02/2026

B2

Question 1 (10 points)

Run-Length Encoding (RLE) is a simple, lossless data compression algorithm that reduces file size by replacing consecutive identical digits (a "run") with a single instance of the digit and a count of its repetitions. For example, the number 2222779 is encoded as 247291 because:

- 2 is repeated 4 times $\rightarrow$ 24
- 7 is repeated 2 times $\rightarrow$ 72
- 9 is repeated 1 time $\rightarrow$ 91

Write the `run_length_encoding` subroutine in ARM assembly language. You must follow this prototype:

```
uint32_t run_length_encoding(uint32_t digits);
```

The subroutine receives an unsigned integer in input, and returns the encoded sequence of the run-length-encoding algorithm as an unsigned integer.

You can assume that:

- each digit in the input number occurs at most 9 times
- no overflow occurs during the computation
- the encoded sequence can be held in a 32 bit unsigned integer.

Suggestions. You can extract the digits of a number from least significant to most significant by repeatedly dividing by 10 and taking the remainder. For example:

$2222779 / 10 = 222277$ with remainder 9

$222277 / 10 = 22227$ with remainder 7

$22227 / 10 = 2222$ with remainder 7

$2222 / 10 = 222$ with remainder 2

$222 / 10 = 22$ with remainder 2

$22 / 10 = 2$ with remainder 2

$2 / 10 = 0$ with remainder 2

You know that there are no more digits when the quotient (result of the division) is 0.

Note that the digits are extracted from right to left; then, they must be processed from left to right in order to compute the Run-Length Encoding. For each group of consecutive identical digits, first append the digit and then the count by repeated multiplication by 10. In detail, you start by initializing $n = 0$ and in every new iteration $n_{new} = n_{old} * 10 + \text{digit}$ and then $n_{new} = n_{old} * 10 + \text{count}$. In the example:

- the digit 2 occurs four times:
 - $n = 0 * 10 + 2 = 2$
 - $n = 2 * 10 + 2 = 24$
- the digit 7 occurs twice:
 - $n = 24 * 10 + 7 = 247$
 - $n = 247 * 10 + 7 = 2472$
- the digit 9 occurs once:
 - $n = 2472 * 10 + 9 = 24729$
 - $n = 24729 * 10 + 1 = 247291$

Question 2 (8 points)

Add the following functionalities to your project.

1) Convert the analog value of the potentiometer to a digital value using the A/D converter. The A/D converter produces a 12-bit digital output. Display the 8 most significant bits of this result using the LEDs. LED 4 corresponds to the most significant bit, and LED 11 corresponds to the least significant bit (of the displayed 8-bit value).

2) When the user presses the *Key1* button, take the last 8 most significant bits produced in the last A/D conversion (i.e., the 8-bit value displayed on the LEDs) and pass this value to the `run_length_encoding` subroutine. Then, display the 8 least significant bits of the result on the LEDs. LED 4 corresponds to the most significant bit, and LED 11 corresponds to the least significant bit of the displayed result.